p1192R1
Experience report - integrating Executors with Parallel Algorithms

Published Proposal, 5 November 2018

Author:

Thomas Rodgers (RedHat)

Audience:

SG1, LEWG

Project:

ISO JTC1/SC22/WG21: Programming Language C++

Abstract

This paper explores integrating Executors with an implementation of the standard parallel algorithms

Table of Contents

§ 1. Background

This paper explores adapting an existing implementation [pstl] of C++17 parallel algorithms to use the executors proposals as outlined in [p0443] and [p1019].

§ 2. Plumbing .on()

[p1019] extends the existing parallel execution policies to take a .on() method which allows an executor to be passed into a standard algorithm taking an exeuction policy. The .on() method rebinds the execution policy to the supplied Executor, e.g.

```
template<typename _ExecutionRequirement, typename _Executor>
class __basic_executor_policy_t : __policy_t<_ExecutionRequirement>
{
    _Executor _M_executor;

    template<typename _tp>
    friend auto __make_execution_policy(_tp __ex) -> __basic_executor_polic
    { return __basic_executor_policy_t<_ExecutionRequirement, _tp>{ std::mo

public:
    _Executor executor() const
    { return _M_executor; }
};

// 2.5, Parallel execution policy
template<typename _ExecutionRequirement, typename _Executor>
class __parallel_policy_t
{
    _Executor _M_executor;

public:
    static constexpr _ExecutionRequirement execution_requirement{};

    static constexpr auto __allow_unsequenced() { return execution_requirem
    static constexpr auto __allow_vector() { return execution_requirement._
    static constexpr auto __allow_parallel() { return execution_requirement

    __parallel_policy_t() = default;

    template<class _Tp>
    auto on(_Tp __ex) const ->
        __basic_executor_policy_t<_ExecutionRequirement, _Tp>
    { return __make_execution_policy(std::move(__ex)); }

    _Executor executor() const
    { return _M_executor; }
};

template<class _Executor>
using parallel_policy_t = __parallel_policy_t<bulk_guarantee_t::parallel_t,
```

```
template<class _Executor>
using parallel_unsequenced_policy_t = __parallel_policy_t<bulk_guarantee_t:
```

Intel's existing implementation relies on Intel's Thread Building Blocks [tbb] library to implement parallel execution. The implementation, however, envisions that this is a pluggable compile time option currently only for implementing std::execution::parallel_policy and std::execution::parallel_unsequenced_policy.

The implementation expects any replacement parallel backend to provide the following functions -

```
void cancel_execution();

template<class Index, class Fp>
void parallel_for( Index first, Index last, Fp f );

template<class Value, class Index, class RealBody, class Reduction>
Value parallel_reduce( Index first, Index last, const Value& identity,
                       const RealBody& realbody, const Reduction& reductior

template<class Index, class Up, class Tp, class Cp, class Rp>
Tp parallel_transform_reduce( Index first, Index last,
                              Up u, Tp init, Cp combine, Rp brickreduce );

template<class Index, class Up, class Tp, class Cp, class Rp, class Sp>
Tp parallel_transform_scan( Index n, Up u, Tp init, Cp combine,
                            Rp brickreduce, Sp scan );

template<class Index, class Tp, class Rp, class Cp, class Sp, class Ap>
void parallel_strict_scan( Index n, Tp initial, Rp reduce, Cp combine,
                           Sp scan, Ap apex );

template<class RandomAccessIterator, class Compare, class LeafSort>
void parallel_stable_sort( RandomAccessIterator xs, RandomAccessIterator xe
                           Compare comp, LeafSort leafsort );

template<class RandomAccessIterator1, class RandomAccessIterator2,
         class RandomAccessIterator3,
         class Compare, class LeafMerge>
void parallel_merge( RandomAccessIterator1 xs, RandomAccessIterator1 xe,
                     RandomAccessIterator2 ys, RandomAccessIterator2 ye,
                     RandomAccessIterator3 zs,
                     Compare comp, LeafMerge leafmerge );

template<class RandomAccessIterator, class Compare>
void parallel_partial_sort( RandomAccessIterator xs, RandomAccessIterator x
                            RandomAccessIterator xe, Compare comp );
```

None of these functions take an execution policy, and thus no way to communicate the executor supplied to .on(). There needs to be a way to carry the execution policy from the user visible library API to the backend. The obvious change to call signatures is to accept the execution policy as the first parameter is explored in this paper, eg.

```
template<class ExecutionPolicy, class Index, class Up, class Tp, class Cp,
Tp parallel_transform_reduce( ExecutionPolicy&& exec,
                              Index first, Index last,
                              Up u, Tp init, Cp combine, Rp brickreduce );
```

Obviously the implementation layer above the backend needs to be modified to pass through the execution policy to the backend as well -

```
template<class _ExecutionPolicy, class _ForwardIterator, class _Function>
__pstl::internal::enable_if_execution_policy<_ExecutionPolicy, void>
for_each(_ExecutionPolicy&& __exec, _ForwardIterator __first, _ForwardItera
    using namespace __pstl;
    internal::pattern_walk1(std::forward<_ExecutionPolicy>(__exec),
                            __first, __last, __f,
                            internal::is_vectorization_preferred<_Executior
                            internal::is_parallelization_preferred<_Executi
}

template<class _ExecutionPolicy, class _ForwardIterator, class _Function, c
void pattern_walk1( _ExecutionPolicy&&, _ForwardIterator __first, _ForwardI
                    /*parallel=*/std::false_type ) noexcept {
    internal::brick_walk1( __first, __last, __f, __is_vector );
}

template<class _ExecutionPolicy, class _ForwardIterator, class _Function, c
void pattern_walk1( _ExecutionPolicy&& __exec, _ForwardIterator __first, _F
                    /*parallel=*/std::true_type ) {
    internal::except_handler([=]() {
        par_backend::parallel_for( std::forward<_ExecutionPolicy>(__exec),
                                   __first, __last,
                                   [__f,__is_vector](_ForwardIterator __i,
                                       internal::brick_walk1( __i, __j, __f
                                   });
        });
}
```

Intel's implementation never conceived of execution policies being used for anything other than tag dispatch types, which given the library's design approach, makes this change a fairly sprawling one to apply. The p1019 approach otherwise fits well within the existing implementation.

§ 3. Digging into the parallel backend

As noted in the preceding section, the [pstl] parallel code requires any replacement backend to provide the following capabilities -

§ 3.1. cancel_execution

Is used to eagerly cancel execution, used by algorithms like find_if. The implementation is fairly straight forward -

```
// Wrapper for tbb::task
inline void cancel_execution() {
    tbb::task::self().group()->cancel_group_execution();
}
```

Every other backend entry point establishes an isolated task group using the following pattern -

```
tbb::this_task_arena::isolate([=]() {
    // interesting bits go here...
});
```

Currently task cancellation is *not* covered in either [p0443] or [p1019]. It is also not strictly required of any backend to support task cancellation. In that case, those algorithms like find_if that might benefit from early termination would just potentially do more work.

§ 3.2. parallel_for

Delegates directly to the tbb supplied implementation -

```
//! Evaluation of brick f[i,j) for each subrange [i,j) of [first,last)
// wrapper over tbb::parallel_for
template<class _Index, class _Fp>
void parallel_for(_Index __first, _Index __last, _Fp __f) {
    tbb::this_task_arena::isolate([=]() {
        tbb::parallel_for(tbb::blocked_range<_Index>(__first, __last), para
    });
}
```

As with TBB, [p0443] has direct support for parallel_for in the form of Bulk one-way execution with blocking semantics required() on .execute(). Mapping the executor modified pstl backend parallel_for to the appropriate execute() call is a simple exercise (TODO provide example).

It is out of scope for executors to provide more sophisticated control structures, so all remaining backend APIs must supply control structures which may be written in terms of [p0443].

§ 3.3. parallel_reduce

Delegates directly to the tbb supplied implementation -

```
//! Evaluation of brick f[i,j) for each subrange [i,j) of [first,last)
// wrapper over tbb::parallel_reduce
template<class _Value, class _Index, typename _RealBody, typename _Reductic
_Value parallel_reduce(_Index __first, _Index __last, const _Value& __ident
                       const _Reduction& __reduction) {
    return tbb::this_task_arena::isolate([__first, __last, &__identity, &__
        return tbb::parallel_reduce(tbb::blocked_range<_Index>(__first, __l
            [__real_body](const tbb::blocked_range<_Index>& __r, const _Val
            return __real_body(__r.begin(),__r.end(), __value);
        },
        __reduction);
    });
}
```

A review of the Intel TBB implementation of parallel_reduce suggests that this functionality can be implemented in terms of [p0443]'s relationship_t::fork_t and relationship_t::continuation_t properties.

§ 3.4. parallel_transform_reduce

Is written in terms of parallel_reduce -

```
template<class _Index, class _Up, class _Tp, class _Cp, class _Rp>
_Tp parallel_transform_reduce( _Index __first, _Index __last, _Up __u, _Tp
    par_trans_red_body<_Index, _Up, _Tp, _Cp, _Rp> __body(__u, __init, __co
    // The grain size of 3 is used in order to provide mininum 2 elements f
    tbb::this_task_arena::isolate([__first, __last, &__body]() {
        tbb::parallel_reduce(tbb::blocked_range<_Index>(__first, __last, 3)
    });
    return __body.sum();
}
```

As, such, it is not really a basis operation for the parallel backend. It is however, likely a customization point for an implementation supplied backend.

§ 3.5. parallel_strict_scan

The [pstl] parallel backend provides a full implementation, adapted from [cilk] which makes the following demands of the [tbb] backend -

```
tbb::parallel_invoke(
        [=] {par_backend::upsweep(__i, __k, __tilesize, __r, __tilesize, __
        [=] {par_backend::upsweep(__i + __k, __m - __k, __tilesize, __r + _
);
```

parallel_invoke here simply spawning a tree of tasks and waiting for the sub-tree's completion. This is a generic control structure which can be implemented in terms of [p0443] one way execution with barrier synchronization.

§ 3.6. parallel_transform_scan

Is implemented in terms of [tbb]'s parallel_scan -

```
template<class _Index, class _Up, class _Tp, class _Cp, class _Rp, class _S
_Tp parallel_transform_scan(_Index __n, _Up __u, _Tp __init, _Cp __combine,
    if (__n <= 0)
        return __init;

    trans_scan_body<_Index, _Up, _Tp, _Cp, _Rp, _Sp> __body(__u, __init, __
    auto __range = tbb::blocked_range<_Index>(0, __n);
    tbb::this_task_arena::isolate([__range, &__body]() {
        tbb::parallel_scan(__range, __body);
    });
    return __body.sum();
}
```

It is likely that parallel_transform_scan can be built in terms of the same basis operations as parallel_strict_scan, and thus is is implementable in terms of [p0443].

§ 3.7. parallel_stable_sort

The [pstl] parallel backend provides a full implementation of this algorithm, which makes the following demands of the [tbb] backend -

```
tbb::task* __right = new(tbb::task::allocate_additional_child_of(*parent())
    merge_task(__xm, _M_xe, __ym, _M_ye, __zm, _M_comp, _M_cleanup, _M_leaf
tbb::task::spawn(*__right);
tbb::task::recycle_as_continuation();
```

The task tree dependency tracking in this code would seem to be covered by [p0443]'s relationship_t::fork_t and relationship_t::continuation_t properties.

```
template<typename _RandomAccessIterator, typename _Compare, typename _LeafS
void parallel_stable_sort(_RandomAccessIterator __xs, _RandomAccessIterator
    tbb::this_task_arena::isolate([=]() {
        //sorting based on task tree and parallel merge
        typedef typename std::iterator_traits<_RandomAccessIterator>::value
        if (__xe - __xs > __PSTL_STABLE_SORT_CUT_OFF) {
          par_backend::buffer<_ValueType> __buf(__xe - __xs);
            if (__buf) {
              typedef stable_sort_task<_RandomAccessIterator, _ValueType*,
              tbb::task::spawn_root_and_wait(*new(tbb::task::allocate_root(

                return;
            }
        }
        // Not enough memory available or sort too small - fall back on ser
        __leaf_sort(__xs, __xe, __comp);
    });
}
```

This is [p0443] .execute() on an executor where the blocking property has been required.

§ 3.8. parallel_merge

The [pstl] parallel backend provides a full implementation of this algorithm which makes the same requirements, in terms of basis functions, as parallel_stable_sort.

§ 3.9. parallel_partial_sort

The [pstl] parallel backend provides a full implementation of this algorithm which makes the same requirements, in terms of basis functions, as parallel_stable_sort and parallel_transform_reduce.

§ 4. Summary

After reviewing an existing implementation of C++17 algorithms, with an eye to determining whether the executor design as proposed is sufficient to implement the execution abstractions for that implementation, the author concludes that this is feasible with a reference implementation in progress.

§ References

§ Informative References

[CILK]

Intel Cilk(TM) Plus. URL: https://www.cilkplus.org/

[P0443]

Jared Hoberock, et. al.. A Unified Executors Proposal for C++. May 07, 2018. URL: https://wg21.link/p0443

[P1019]

Jared Hoberock. Integrating Executors with Parallel Algorithms. May 07, 2018. URL: https://wg21.link/p1019

[PSTL]

Intel Parallel STL implementation. URL: https://github.com/intel/parallelstl

[TBB]

Thread Building Blocks (Intel TBB). URL: https://www.threadingbuildingblocks.org/